



**UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJE Y COMPILADORES SECCIÓN 01**

Tema 3: Lenguajes y Gramáticas Formales

Informe de Investigación y Casos Prácticos de Aplicación

Autores:

Fernando Centeno (C.I. 30.810.484)

Juan Longart (C.I. 31.882.343)

Adrian Reina (C.I. 31.317.970)

Rafael Rodríguez (C.I. 31.882.367)

Profesor:

Ing. Félix Márquez

Ciudad Guayana, junio de 2026

1. Introducción

El estudio de los lenguajes de programación requiere ir más allá de la simple escritura práctica del código fuente. Para construir, optimizar e interpretar lenguajes de manera robusta, es imperativo comprender las bases matemáticas e ingenieriles que rigen su generación y su reconocimiento. En este documento se abordan los conceptos nucleares de los alfabetos, palabras y lenguajes formales, la jerarquización y poder computacional a través de la Jerarquía de Chomsky, así como la higiene y optimización sintáctica de las gramáticas.

Adicionalmente, se presentan dos casos prácticos de alta relevancia en la informática: primero, el diseño e implementación de una Gramática Libre de Contexto que modela herramientas gráficas de dibujo a partir de un alfabeto simplificado. Segundo, el desarrollo de un validador sintáctico en base a un Autómata Finito Determinístico para jugadas individuales y secuencias completas en notación PGN de Ajedrez. Estos componentes demuestran cómo la abstracción de problemas reales se traduce en algoritmos eficientes en el desarrollo moderno de software.

2. Desarrollo Técnico y de Investigación

2.1. Fundamentos y Jerarquía (Teoría Aplicada)

2.1.1. Conceptos Básicos y Relación Gramática-Lenguaje

Un alfabeto (denotado comúnmente por Σ) es un conjunto finito y no vacío de símbolos elementales. Una palabra o cadena es una secuencia finita de símbolos tomados de dicho alfabeto. Un lenguaje formal L se define simplemente como cualquier subconjunto del conjunto de todas las palabras posibles generables sobre el alfabeto Σ , incluyendo la cadena vacía (ϵ), denotado por Σ^* . Es decir, $L \subseteq \Sigma^*$.

Para modelar formalmente la estructura y validez sintáctica de estas cadenas, se emplean las gramáticas formales. Matemáticamente, una gramática G es una tupla de cuatro elementos: $G = (V_N, V_T, P, S)$, donde V_N es el conjunto finito de variables sintácticas o no terminales; V_T es el alfabeto de símbolos terminales (los cuales compondrán la cadena final); P es el conjunto de reglas de producción que

dictan los reemplazos; y $S \in V_N$ es el axioma inicial.

El mecanismo mediante el cual la gramática genera el lenguaje es la derivación ($\Rightarrow$). Comenzando desde S , se aplican sucesivas sustituciones de subcadenas según las reglas de producción en P hasta que la cadena resultante conste únicamente de símbolos terminales. El conjunto de todas las cadenas de terminales que pueden ser derivadas de este modo conforma el lenguaje formal generado por la gramática, formalizado como: $L(G) = \{w \in V_T^* \mid S \Rightarrow^* w\}$.

Ejemplo Práctico de Generación:

Supongamos una gramática para expresiones aritméticas básicas: $G = (\{E\}, \{n, +, *\}, P, E)$, con reglas de producción:

1. $E \rightarrow E + E$
2. $E \rightarrow E * E$
3. $E \rightarrow n$

Para generar la cadena ' $n + n * n$ ', realizamos la derivación por la izquierda:

$E \Rightarrow E + E \Rightarrow n + E \Rightarrow n + E * E \Rightarrow n + n * E \Rightarrow n + n * n$.

2.1.2. Jerarquía de Chomsky

La Jerarquía de Chomsky es una clasificación de gramáticas formales propuesta por Noam Chomsky en 1956. Consta de cuatro tipos de gramáticas concéntricas (Tipo 0 a Tipo 3), donde cada tipo impone mayores restricciones a las producciones sintácticas y demanda un modelo computacional diferente para su reconocimiento léxico y sintáctico.

Tipo	Gramática	Reglas de Producción	Autómata Reconocedor
Tipo 0	Sin restricciones	$\alpha \rightarrow \beta$ (sin restricciones)	Máquina de Turing
Tipo 1	Sensible al Contexto	$\alpha \rightarrow \beta$ (donde $ \alpha \leq \beta $)	Autómata Lineal Acotado
Tipo 2	Libre de Contexto	$A \rightarrow \beta$ (donde A es no terminal)	Autómata con Pila
Tipo 3	Regular	$A \rightarrow wB$ o $A \rightarrow w$ (regular der.)	Autómata Finito

Ejemplos prácticos en Notación BNF (Forma de Backus-Naur):

Tipo 0: Gramática sin restricciones

```
<S> ::= <A> <B>
<A> <B> ::= <B> <A>
<A> ::= "a"
<B> ::= "b"
```

Tipo 1: Sensible al contexto (lenguaje $\{a^n b^n c^n\}$)

```
<S> ::= "a" <S> <B> <C> | "a" <B> <C>
<C> <B> ::= <B> <C>
"a" <B> ::= "a" "b"
"b" <B> ::= "b" "b"
"b" <C> ::= "b" "c"
"c" <C> ::= "c" "c"
```

Tipo 2: Libre de contexto (expresiones matemáticas comunes)

```
<expr> ::= <expr> "+" <term> | <term>
<term> ::= <term> "*" <factor> | <factor>
<factor> ::= "(" <expr> ")" | "n"
```

Tipo 3: Regular (definición léxica de identificadores de programación)

```
<identificador> ::= <letra> | <letra> <resto>
<resto> ::= <letra> <resto> | <digito> <resto> | <letra> | <digito>
<letra> ::= "a" | "b" | "c"
<digito> ::= "0" | "1" | "2"
```

2.2. Derivación y Modelado (Caso Práctico: Dibujo)

Se diseñó una Gramática Libre de Contexto (GLC) con el alfabeto estrictamente limitado a $\Sigma = \{a, c, g, t\}$. Esta gramática modela una herramienta de dibujo que interpreta las cadenas generadas mediante un sistema de gráficos de tortuga con pila de estados. Los terminales se mapean de la siguiente forma:

- a: Avanzar una distancia fija dibujando una línea recta en la dirección actual.
- g: Girar la dirección 45 grados en el sentido de las agujas del reloj (hacia la derecha).
- c: Hacer push de las coordenadas e inclinación actuales en una pila de estados.
- t: Hacer pop para restaurar la posición y dirección al último estado almacenado en la pila.

2.2.1. Definición Formal y Reglas

La gramática se define como $G_D = (V_N, V_T, P, S)$, donde $V_N = \{S, D, L, R\}$, $V_T = \{a, c, g, t\}$, y las reglas de producción en P son:

1. $S \rightarrow D$
2. $D \rightarrow a g g a g g a g g a$ (Cuadrado)
3. $D \rightarrow a c L D t c R D t$ (Estructura de Árbol)
4. $D \rightarrow a$ (Parada/Hojas del Árbol)
5. $D \rightarrow c g a t a c g a t g g a c g a t g g a c g a t g g a g a g g g g g g g a g g a g g a g g a$ (Cubo)
6. $L \rightarrow g g g g g g g$ (Turn Left 45°: girar 315° derecha)
7. $R \rightarrow g$ (Turn Right 45°)

2.2.2. Derivaciones y Representaciones Sintácticas

a) Cuadrado:

Derivación: $S \Rightarrow D \Rightarrow a g g a g g a g g a$.

Coche o Cadena final: aggaggaggagga (Giro de 90° en las esquinas mediante 'g g').

b) Árbol con varias ramas y hojas:

Derivación:

1. $S \Rightarrow D$
2. $\square a c L D t c R D t$ (Tronco y ramas primarias)
3. $\square a c L (a c L D t c R D t) t c R D t$ (Expandir rama izquierda)
4. $\square a c L (a c L D t c R D t) t c R (a c L D t c R D t) t$ (Expandir rama derecha)
5. $\square a c L (a c L a t c R a t) t c R (a c L a t c R a t) t$ (Hojas terminales)
6. $\square a c g^7 a c g^7 a t c g a t t c g a c g^7 a t c g a t t$ (Sustituir L y R)

Cadena final: $acgggggggacgggggggatcgattcgacgggggggatcgatt$

c) Cubo en perspectiva:

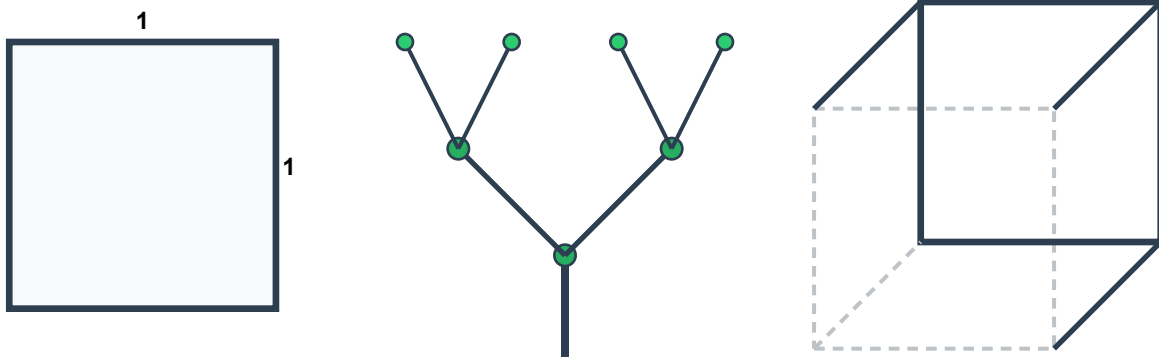
Dibuja el marco frontal del cubo, las 4 aristas oblicuas utilizando la pila de estados, y el marco posterior.

Cadena

final:

$cgatacgatggagggggcgatggggaggggacgatggggggagggaggggggagggaggagga$

Figura 1: Renderizado de las cadenas resultantes (Cuadrado, Árbol y Cubo) generado por la herramienta de interpretación.



2.3. Higiene y Optimización de Gramáticas

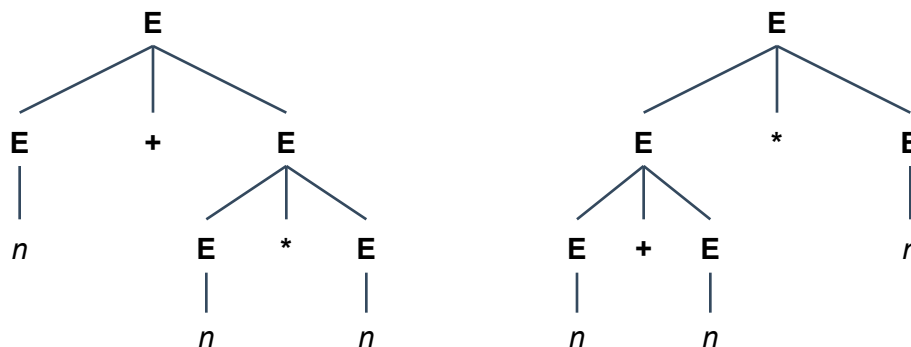
2.3.1. Ambigüedad Sintáctica

Una gramática es ambigua si permite construir dos o más árboles de análisis sintáctico diferentes para una misma palabra del lenguaje. Esto representa un problema grave en compiladores, ya que puede alterar el orden de evaluación semántica (ej. la precedencia aritmética).

Caso Práctico: Gramática ambigua clásica de expresiones matemáticas:

$$E \rightarrow E + E \mid E * E \mid n$$

Para la palabra $n + n * n$, existen dos árboles de derivación diferentes:



Solución: Para solucionar la ambigüedad, se estratifica la gramática de modo que los operadores con mayor precedencia se evalúen a mayor profundidad:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow n$$

2.3.2. Recursividad por la Izquierda Directa

La recursividad por la izquierda directa ocurre cuando un no terminal tiene una producción de la forma $A \rightarrow A\alpha$. Esto causa bucles infinitos en analizadores sintácticos recursivos descendentes LL(1).

Algoritmo de eliminación paso a paso:

Si tenemos las producciones:

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid \beta_1 \mid \beta_2 \mid \dots$$

Se sustituyen introduciendo un no terminal A' :

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \epsilon$$

Ejemplo aplicado: Eliminar recursividad en $E \rightarrow E + T \mid T$

1. Identificamos: $A = E$, $\alpha_1 = +T$, $\beta_1 = T$.

2. Aplicamos la transformación:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

2.3.3. Factorización por la Izquierda

La factorización por la izquierda se realiza cuando dos producciones del mismo no terminal tienen un prefijo común (ej. $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$). Un analizador LL(1) no sabrá cuál elegir mirando un solo símbolo adelante. La factorización pospone la decisión sintáctica.

Algoritmo de Factorización:

Se transforman las reglas compartidas introduciendo una variable A' :

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots$$

Ejemplo aplicado (Dilema de la condicional simple):

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \mid a$$

1. Prefijo común: $\alpha = \text{if } E \text{ then } S$. Restos: $\beta_1 = \text{else } S$, $\beta_2 = \epsilon$. No compartido: $\gamma = a$.

2. Gramática resultante optimizada:

$$S \rightarrow \text{if } E \text{ then } S S' \mid a$$

$$S' \rightarrow \text{else } S \mid \epsilon$$

2.4. De la Expresión al Autómata (Caso Práctico: Ajedrez)

Se diseñó y programó un reconocedor sintáctico para un subconjunto simplificado del estándar PGN de Ajedrez, el cual describe jugadas elementales de piezas y peones, capturas simplificadas (ej. xd5) y estados de jaque o jaque mate.

2.4.1. Expresión Regular Equivalente

El subconjunto de jugadas válidas se especifica bajo la expresión regular:

Regex: `[KQRBN]?x?[a-h][1-8][+ #]?`

2.4.2. Definición Formal del Autómata (AFD)

El Autómata Finito Determinístico equivalente se define formalmente como $M = (Q, \Sigma, \delta, q_0, F)$ donde:

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_e\}$
- $\Sigma = \{K, Q, R, B, N, x, a..h, 1..8, +, \#\}$
- q_0 : Estado inicial.
- $F = \{q_4, q_5\}$: Estados de aceptación (movimiento básico completo o movimiento con jaque/mate).
- q_e : Estado muerto de error.

Tabla de Transiciones de Estado del AFD:

Estado Actual	{K,Q,R,B,N}	{x}	{a-h}	{1-8}	{+,#}	Otro
q0 (Inicio)	q1	q2	q3	qe	qe	qe
q1 (Pieza)	qe	q2	q3	qe	qe	qe
q2 (Captura)	qe	qe	q3	qe	qe	qe
q3 (Columna)	qe	qe	qe	q4	qe	qe
q4 (Fila) *	qe	qe	qe	qe	q5	qe
q5 (Jaque) *	qe	qe	qe	qe	qe	qe

2.4.3. Implementación Algorítmica

Se programó una clase en Python que encapsula la lógica de transición del autómata, validando cadenas individuales y secuencias completas de jugadas PGN. A continuación, se adjunta el segmento fundamental del validador:

```
class PGNValidator:
    def transition(self, state, char):
        if state == 0:
            if char in {'K', 'Q', 'R', 'B', 'N'}: return 1
            elif char == 'x': return 2
            elif 'a' <= char <= 'h': return 3
        elif state == 1:
            if char == 'x': return 2
            elif 'a' <= char <= 'h': return 3
        elif state == 2:
            if 'a' <= char <= 'h': return 3
        elif state == 3:
            if '1' <= char <= '8': return 4
        elif state == 4:
            if char in {'+', '#'}: return 5
        return 99 # Estado de error
```

Referencias Bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2da ed.). Pearson Educación.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113-124.
<https://doi.org/10.1109/TIT.1956.1056813>

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3ra ed.). Pearson Educación.

Pratt, T. W., & Zelkowitz, M. V. (2001). *Lenguajes de programación: Diseño e implementación* (4ta ed.). Prentice Hall.